Lab Assignment: Readers-Writers Problem using Monitor in Python

Course: COMPX234 | Assessment: Lab Assignment 2 | Total: 100 Marks (5%)

Assignment type	Individual laboratory task
Programming language	Python 3 using threading and condition variables
Submission	GitHub repository link + local demonstration in lab
Academic integrity	Your own original work only (Any AI assistance must be acknowledged, and students must be able to explain all submitted code in the lab demo.)

Objective: Implement a monitor-style solution for the Readers-Writers problem in Python. Your program must allow multiple readers to read at the same time when no writer is writing, while writers must get exclusive access to the shared resource.

```
monitor ReadersWriters {
    int readers = 0; writers = 0;
    condition canRead, canWrite;

    procedure StartRead() {
        if (writers > 0) wait(canRead);
        readers++;
        signal(canRead);
    }

    procedure EndRead() {
        readers--;
        if (readers == 0) signal(canWrite);
    }

    procedure StartWrite() {
        if (readers > 0 || writers > 0) wait(canWrite);
        writers++;
    }

    procedure EndWrite() {
        writers--;
        if (readers > 0) signal(canRead);
        else signal(canWrite);
    }
}
```

Reference pseudocode provided with the lab brief

1. Learning outcomes

- Explain the difference between readers and writers in a shared data access problem.
- Use Python threading primitives to build a monitor-like synchronization solution.
- Control concurrent access using shared state, condition variables, and safe signaling.
- Test concurrent code and interpret execution logs from multiple threads.
- Use GitHub for version control, commit history, and final submission.

2. Background

The Readers-Writers problem is a classic synchronization problem in operating systems. A shared resource, such as a file, database record, or memory buffer, is accessed by two types of threads:

- Reader threads only read data. Many readers may read at the same time.
- Writer threads change data. Only one writer may write at a time, and no reader may read while a writer is writing.

In Java, this task is often introduced using monitor-style synchronization with synchronized methods, wait(), and notifyAll(). In Python, you will build the same idea by using a class that protects shared state with threading. Lock and threading.Condition. This class will act as a monitor.

3. Problem statement

Write a Python program that simulates multiple reader threads and writer threads accessing one shared resource. Your monitor must enforce these rules:

- If a writer is writing, no reader may start reading.
- If one or more readers are reading, no writer may start writing.
- Multiple readers may read together when no writer is active.
- The monitor must correctly update counters and wake waiting threads when the state changes.

You may use a readers-preference or neutral monitor design, but your implementation must be correct, easy to read, and clearly documented in code comments. The starter file provided with this assignment is based on the pseudocode shown above.

4. Task requirements

What is already provided in the starter code

Do not modify the Reader and Writer classes. You may complete only the TODO sections in main() if required by the starter file.

- `Reader` class (thread behavior for readers - fully implemented — do not change)

- `Writer` class (thread behavior for writers - fully implemented — do not change)
- `main()` function (partially provided)
- Simulation logic using `time.sleep()` and print statements

What you must complete

1. Complete the monitor class named ReadersWritersMonitor.
2. Implement the following four methods:
    - start_read()
    - end_read()
    - start_write()
    - end_write()
3. Use Python Condition (threading.Condition) to:
    - make threads wait when they cannot proceed
    - wake up waiting threads when the monitor state changes
4. Complete the TODO sections in main() to:
    - create at least 3 reader threads
    - create at least 2 writer threads
    - start all threads
    - wait for all threads to finish
    - print a final completion message

Program Requirements

1. Ensure the following rules are satisfied:
    - Multiple readers can read at the same time
    - Writers must have exclusive access
2. Use `time.sleep()` with small random values to simulate reading and writing.
3. Print clear messages to show:
    - when a reader starts and stops reading
    - when a writer starts and stops writing
4. Run the program and verify that:
    - readers can run together
    - writers do not run with readers or other writers

Required output messages

Your exact wording may differ, but the output should clearly show when a reader or writer is waiting, starts, and finishes. Example messages are shown below.

```
Reader 1 is waiting to read
Reader 1 starts reading. Active readers = 1
Reader 2 starts reading. Active readers = 2
Reader 1 stops reading. Active readers = 1
Writer 1 starts writing
Writer 1 stops writing
```

5. Files to create in your GitHub repository

- readers_writers.py - your completed program
- README.md - short explanation of your design, how to run the program, and one sample output
- Optional: screenshots/ or output.txt if you want to include evidence of execution

You may start from the provided starter file and rename it to readers_writers.py after you complete the TODO sections.

6. GitHub submission instructions

1. Create a new GitHub repository for this lab. Suggested repository name: compx234-readers-writers-lab2.
2. Add your Python file and README.md to the repository.
3. Make regular and meaningful commits during your work. A good submission will normally show around 6-10 meaningful commits such as: initial starter setup, implemented monitor logic, final testing and cleanup.
4. Push your work to GitHub before the end of the lab.
5. Submit the GitHub repository link to your Canvas.
6. During the lab demo, be ready to explain your monitor logic and run the program.

7. Important implementation notes

- Use while loops around condition waits. Do not use only if before waiting, because a thread must re-check the condition after waking up.
- Protect shared counters such as active_readers and active_writers inside the monitor only.
- Use notify_all() when waking waiting threads if that makes your design simpler and safer.
- Keep your code readable. Use comments to explain why a thread waits and when it may continue.
- Test your program more than once because thread scheduling changes from run to run.

8. Marking scheme (100 marks)

Criterion	Marks	What is expected
Correct monitor logic	30	Readers and writers are synchronized correctly; shared state is safe.
Proper use of condition variables	15	Waiting and signaling are used correctly with re-checking conditions.
Thread implementation and simulation	15	Multiple reader and writer threads are created and executed correctly.

Program output and testing	10	Logs clearly show valid behavior and demonstrate concurrent execution.
Code quality and comments	10	Readable code, meaningful names, and comments that explain the design.
GitHub usage and commit history	20	Repository is properly organized with meaningful commits.

9. Submission checklist

- □ My Python file runs without syntax errors.
- □ My monitor uses condition variables and shared counters safely.
- □ My output shows multiple readers can read together.
- □ My output never allows a writer and a reader at the same time.
- □ My GitHub repository contains my code and README.md.
- □ I made meaningful commits and pushed my final version.
- □ I am ready to demonstrate the program in the lab.

10. Starter code guidance

A starter Python file is provided with this assignment. It includes the monitor class structure, thread classes, and TODO sections. You must complete the synchronization logic, test it, and commit your final solution to GitHub.

Good luck. Focus on correctness first, then improve readability and comments.